DEWIZ BLOCKCHAIN

Architecture Decision Record

ADR-002

Privacy-Preserving x402 Payments

Integrating ERC-5564 Stealth Addresses with the HTTP 402Payment Standard for Stablecoin Transactions

Status	PROPOSED
Date	2026-04-07
Author	Jeff Prestes
Team	Dewiz Blockchain
Supersedes	None

CONFIDENTIAL

1. Context

1.1 The x402 Payment Protocol

The x402 protocol, launched by Coinbase in May 2025, operationalizes the long-reserved HTTP 402 “Payment Required” status code to enable instant stablecoin payments directly over HTTP. It creates a standardized pay-per-request loop: a server responds with 402 and machine-readable payment requirements (price, token, network, destination address), the client signs a payment authorization (typically via EIP-712), retries the request with an X-PAYMENT header, and a facilitator verifies and settles the payment on-chain.

x402 is designed for autonomous AI agents, API monetization, and machine-to-machine commerce. It eliminates the need for API keys, subscriptions, or manual payment processing, supporting USDC and EURC natively via EIP-3009, with extensibility to any ERC-20 token through Permit2 and SPL tokens on Solana.

The protocol is backed by Cloudflare, AWS, Anthropic, Circle, NEAR, Google (via A2A extension), and Vercel, among others. Settlement occurs on Layer 2 networks (primarily Base) with sub-second finality and near-zero fees.

1.2 The Privacy Gap in x402

While x402 solves the payment UX problem for the internet, it introduces a significant privacy gap: every payment creates a permanent, publicly visible on-chain link between the payer’s wallet and the resource server’s receiving address. This creates several risks:

- **Revenue Exposure:** API providers accumulate all payments to a static wallet, revealing total revenue, client count, and usage patterns to any block explorer.
- **Behavioral Profiling:** AI agents making hundreds of API calls per day from the same wallet create rich behavioral profiles susceptible to chain analysis clustering heuristics (co-spend, deposit address, gas price fingerprinting).
- **Identity Linkage:** x402 links web2 metadata (IP addresses, timestamps, API endpoints) to web3 transactions, creating traceable patterns that undermine pseudonymity.
- **Competitive Intelligence:** Competitors can monitor an API provider’s payment wallet to infer client relationships, pricing tiers, and business trajectory.

1.3 ERC-5564 Stealth Addresses

Dewiz Blockchain has implemented a Rust library (dewiz-stealth-address) providing the full ERC-5564 stealth address lifecycle: key generation, stealth address derivation, announcement scanning, private key recovery, and wallet creation. The implementation uses Scheme ID 1 (secp256k1 with view tags) and integrates with the alloy Ethereum toolkit.

ERC-5564 stealth addresses solve recipient privacy by generating a fresh, unlinkable one-time address for each payment. The cryptographic scheme ensures that even with full blockchain visibility, an observer cannot link two stealth addresses belonging to the same recipient, identify the recipient of any given payment, or aggregate the recipient’s total balance across stealth addresses.

Two singleton contracts are deployed across 7 mainnets and 5 testnets via CREATE2: the ERC-5564 Announcer (0x5564...5564) for publishing payment announcements, and the ERC-6538 Registry (0x6538...6538) for recipients to register their stealth meta-addresses.

2. Decision

We will integrate the dewiz-stealth-address library into the x402 payment flow to provide privacy-preserving stablecoin payments. Specifically:

1. **Build x402-stealth-middleware:** A server-side middleware (Rust crate with Node.js/Python FFI bindings) that replaces the static payTo address in x402's 402 response with a freshly generated stealth address per request.
2. **Extend the x402 PaymentRequired schema:** Add an optional stealthMetaAddress field and ephemeralPubKey field to the 402 response JSON, enabling clients that support stealth addresses to derive them client-side as well.
3. **Implement a Stealth-Aware Facilitator:** A facilitator service that, after settling the payment on-chain, also calls the ERC-5564 Announcer contract to emit the announcement event, enabling the recipient to scan and recover funds.
4. **Build a Stealth Scanner Service:** A background daemon that uses the viewing key to continuously scan ERC-5564 announcement events, detect incoming payments, and aggregate balances across stealth addresses for the API provider.
5. **Integrate ERC-4337 Paymaster for gas sponsorship:** Stealth addresses start with zero ETH; a paymaster sponsors gas for sweeping funds from stealth wallets, preventing the funding problem from re-linking addresses.

3. Architecture

3.1 Payment Flow (Privacy-Preserving x402)

The following describes the complete request-to-settlement flow with stealth address integration:

Step	Actor	Action
1	**Client**	Sends standard HTTP GET /api/resource to the resource server.
2	**Server**	Determines payment is required. Calls generate_stealth_address(K, V) from dewiz-stealth-address to derive a fresh one-time address. Returns HTTP 402 with PaymentRequired JSON including: stealth payTo address, ephemeralPubKey (R), viewTag, price, token (USDC), and network (Base).
3	**Client**	Receives 402 response. Signs a payment authorization for the stealth address using EIP-712 (or EIP-3009 TransferWithAuthorization for USDC). Retries the request with X-PAYMENT header containing the signed payload.
4	**Server**	Forwards the payment payload to the Stealth-Aware Facilitator for verification.

5	**Facilitator**	Verifies the signed authorization. Settles the USDC transfer to the stealth address on-chain. Calls ERC5564Announcer.announce(schemeId=1, stealthAddress, ephemeralPubKey, metadata) to emit the announcement event. Returns settlement confirmation to the server.
6	**Server**	Receives confirmation. Returns HTTP 200 with the requested resource and X-PAYMENT-RESPONSE header.
7	**Scanner**	Background daemon uses the recipient's viewing key (v) to scan announcement events. Computes shared secret S' = v·R, hashes to scalar s', checks view tag (filters 99.6% of non-matching events). On match, derives stealth private key p = k + s' mod n.
8	**Sweeper**	Periodically consolidates funds from stealth wallets using ERC-4337 UserOperations with paymaster gas sponsorship, routing through a mixer or batched settlement to avoid re-linkage.

3.2 Component Architecture

Component	Language	Responsibility
x402-stealth-middleware	Rust + FFI	Intercepts incoming requests, generates stealth addresses per-request, enriches 402 responses with ephemeral key material, manages stealth meta-address lifecycle.
dewiz-stealth-address	Rust	Core cryptographic library: key generation, stealth address derivation (secp256k1 + view tags), announcement scanning, private key recovery. Already implemented.
Stealth-Aware Facilitator	Rust / Node.js	Extends x402 facilitator to verify payment, settle on-chain, AND emit ERC-5564 announcement. Can wrap Coinbase's facilitator or operate standalone.
Stealth Scanner Daemon	Rust	Continuously monitors ERC-5564 Announcer events using viewing key. Detects payments, indexes stealth addresses, tracks balances. Exposes REST API for balance queries.
ERC-4337 Sweeper	Rust / Solidity	Constructs UserOperations to move funds from stealth wallets via paymaster-sponsored gas. Batches and routes through privacy-preserving settlement paths.
ERC-6538 Registry Client	Rust	Manages registration and retrieval of stealth meta-addresses from the on-chain Registry singleton. Used during server startup and key rotation.

3.3 On-Chain Contract Interactions

Contract	Address	Usage in Flow

ERC-5564 Announcer	0x5564...5564	Facilitator calls announce() after settling payment to emit (R, tag, stealthAddr) for scanner discovery.
ERC-6538 Registry	0x6538...6538	API provider registers stealth meta-address at startup via registerKeys(). Clients can also read meta-address to derive stealth addresses client-side.
USDC (EIP-3009)	Network-specific	TransferWithAuthorization enables gasless signed transfers to stealth addresses without the sender needing to execute an on-chain transaction.
ERC-4337 EntryPoint	Network-specific	Processes UserOperations from stealth wallets for fund sweeping with paymaster gas sponsorship.

3.4 Extended x402 PaymentRequired Schema

The standard x402 PaymentRequired response is extended with optional stealth-address fields:

```
{
  "x402Version": 2,
  "schemes": [{
    "scheme": "exact",
    "network": "base",
    "maxAmountRequired": "1000000",
    "resource": "https://api.example.com/data",
    "description": "API call: /data",
    "mimeType": "application/json",
    "payTo": "0x<stealth_address>",
    "token": "0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913",
    "extra": {
      "stealth": {
        "schemeId": 1,
        "ephemeralPubKey": "0x02<33_bytes_compressed_R>",
        "viewTag": "0xa7",
        "registryAddress": "0x6538...6538",
        "announcerAddress": "0x5564...5564"
      }
    }
  }]
}
```

4. Privacy Properties Achieved

Property	Without Stealth (x402 only)	With Stealth Integration
Unlinkability	All payments to same address — trivially linked	Each payment to unique stealth address — cryptographically unlinkable
Untraceability	Recipient identity visible via payTo address	Observer cannot identify recipient from stealth address
Balance Hiding	Total revenue visible by summing wallet balance	Revenue fragmented across unlinkable stealth wallets
Pattern Hiding	Usage patterns (frequency, timing, amounts) fully exposed	Patterns broken across independent addresses
Sender Privacy	Sender wallet exposed for all API calls	Can be combined with client-side stealth wallets + paymaster
Clustering Resistance	Vulnerable to co-spend, deposit address, gas price heuristics	Stealth addresses + paymaster break all major clustering vectors

5. Alternatives Considered

5.1 Tornado Cash / Privacy Mixers

Rejected. Tornado Cash is sanctioned by OFAC (August 2022). Using it would make the solution non-compliant for institutional clients, which contradicts Dewiz's focus on regulatory compliance (OFAC, SEC, ECB/MiCA hooks in Token Factory). Additionally, mixers require pre-funded pools and introduce latency incompatible with x402's sub-second settlement expectations.

5.2 Railgun Privacy Protocol

Considered but deferred. Railgun uses zkSNARKs for shielded transfers and maintains its own UTXO-based balance system. While cryptographically stronger (full zero-knowledge proofs), it requires significant protocol-level integration, adds complexity to the x402 flow, and introduces a dependency on Railgun's relayer infrastructure. Stealth addresses are a lighter-weight solution that integrates directly at the HTTP/EVM layer without additional protocol dependencies.

5.3 Static Rotating Addresses (No Cryptographic Guarantees)

Rejected. Simply generating new addresses from an HD wallet provides no cryptographic unlinkability. An observer can trivially link addresses through on-chain analysis (funding patterns, consolidation transactions, common gas source). ERC-5564 provides mathematically proven unlinkability through ECDH shared secrets.

5.4 Client-Side Only (No Server Integration)

Partially adopted. Client-side stealth address generation is supported as an optional enhancement (clients can derive stealth addresses from the meta-address published in the 402 response). However, server-side generation is the primary approach because: the server controls the 402 response and can guarantee every payment goes to a unique address; it requires no client-side changes for existing x402 clients; and the announcement emission can be coupled with settlement in the facilitator.

6. Target Networks

Network	Priority	x402 Support	ERC-5564 Deployed
Base	P0 (Primary)	Native (Coinbase)	Yes — Mainnet + Sepolia
Ethereum	P1	Yes	Yes — Mainnet + Sepolia + Holesky
Arbitrum	P1	Yes	Yes — Mainnet + Sepolia
Optimism	P2	Yes	Yes — Mainnet + Sepolia
Gnosis Chain	P2	Community	Yes — Mainnet
Solana	P3 (Future)	Yes (SPL)	Requires separate implementation

7. Implementation Plan

Phase	Deliverable	Duration	Dependencies
Phase 1	x402-stealth-middleware crate with Node.js FFI bindings; integration tests on Base Sepolia	4 weeks	dewiz-stealth-address (done)
Phase 2	Stealth-Aware Facilitator (wrapping Coinbase facilitator + ERC-5564 announce call)	3 weeks	Phase 1; Coinbase x402 SDK
Phase 3	Stealth Scanner Daemon with REST API for balance tracking and payment indexing	3 weeks	Phase 1
Phase 4	ERC-4337 Sweeper with paymaster integration for gas-sponsored fund consolidation	4 weeks	Phase 3; ERC-4337 bundler
Phase 5	End-to-end integration testing on Base mainnet; security audit of cryptographic flow	3 weeks	Phases 1–4

Phase 6	x402 schema extension proposal (PR to coinbase/x402 repo) and documentation	2 weeks	Phase 5

8. Risks and Mitigations

Risk	Severity	Mitigation
Funding Problem (re-linkage when sweeping stealth wallets)	High	ERC-4337 paymaster sponsors gas; sweeper batches and routes through privacy-preserving paths; consolidation uses different timing and amounts to break patterns.
x402 Schema Adoption (upstream acceptance of stealth fields)	Medium	Stealth fields are in the “extra” object (already extensible by design). Can function without upstream changes. PR to coinbase/x402 is a nice-to-have, not a blocker.
View Tag Collision (false positives in scanner)	Low	View tags provide 1-byte (256-value) optimization, filtering 99.6% of non-matching events. Full address verification eliminates false positives. Performance cost is minimal.
Facilitator Centralization	Medium	Initial deployment wraps Coinbase facilitator; Phase 5 includes standalone facilitator option for decentralized operation. The Stealth-Aware Facilitator can also be self-hosted.
Regulatory Compliance Tension	Medium	Stealth addresses provide privacy, not anonymity. The viewing key enables selective disclosure to authorized parties (regulators, auditors). This aligns with Dewiz’s compliance-first approach (OFAC/SEC/MiCA hooks).
Gas Costs for Announcements	Low	ERC-5564 announce() is a lightweight event emission (~50k gas on L2). On Base, this costs <$0.01 per announcement. Bundling with the settlement transaction amortizes further.

9. Compliance and Selective Disclosure

A critical design principle of this integration is that stealth addresses provide privacy against public observers, not anonymity from authorized parties. The viewing key architecture of ERC-5564 enables a compliance-compatible privacy model:

- **Selective Disclosure:** The API provider can share their viewing key with regulators or auditors, enabling them to scan all incoming payments without revealing the spending key (which controls fund movement).
- **Audit Trail:** The Stealth Scanner Daemon maintains a complete internal ledger of all detected payments, mapping stealth addresses to payment contexts (API endpoint, timestamp, amount). This ledger is available for compliance reporting without being publicly visible.

- **Token Factory Integration:** For stablecoins deployed via Dewiz's Token Factory platform (which includes OFAC, SEC, and ECB/MiCA regulatory hooks), the compliance hooks can be extended to validate stealth address payments against allowlists and denylist registries before settlement.
- **ERC-8004 Identity Integration:** The stealth meta-address can be linked to an ERC-8004 Identity Registry entry (already deployed on Base Sepolia by Dewiz), enabling KYC-verified entities to use stealth addresses while maintaining regulatory compliance through the identity layer.

10. Key Dependencies

Dependency	Version / Status	Notes
dewiz-stealth-address	Current (Rust)	Core library, already implemented. Provides all cryptographic primitives.
alloy	1.7.3+	Ethereum toolkit for RPC calls, contract bindings, wallet management.
k256	Latest	secp256k1 curve arithmetic (ProjectivePoint, Scalar, NonZeroScalar).
Coinbase x402 SDK	V2	Server and client SDK for x402 protocol. Node.js, Python, Go.
ERC-5564 Announcer	Deployed (7 chains)	Singleton at 0x5564...5564. No deployment needed.
ERC-6538 Registry	Deployed (7 chains)	Singleton at 0x6538...6538. No deployment needed.
ERC-4337 Infrastructure	Bundler + Paymaster	Required for Phase 4. Can use existing bundler services (Pimlico, Stackup, etc.).

11. Consequences

11.1 Positive

- API providers gain cryptographically proven recipient privacy for all x402 payments.
- AI agents can make payments without building a traceable behavioral profile on-chain.
- The solution is backward-compatible: existing x402 clients that don't understand stealth fields simply pay to the stealth address as if it were any other EOA.
- Dewiz positions itself at the intersection of privacy and compliance, a unique market position in the institutional blockchain space.
- Leverages existing deployed infrastructure (ERC-5564/6538 singletons on 7 mainnets) with no new contract deployment required.

11.2 Negative / Trade-offs

- Increased operational complexity: the scanner daemon, sweeper, and facilitator extension add moving parts to the infrastructure.
- The funding problem (gas for stealth wallets) requires ERC-4337 paymaster integration, adding a dependency on account abstraction infrastructure.
- Per-request stealth address generation adds ~2ms of computational overhead (secp256k1 point multiplication) to the 402 response path.
- Stealth address scanning requires continuous event monitoring, which introduces infrastructure costs proportional to announcement volume.

12. References

- ERC-5564: Stealth Addresses
- ERC-6538: Stealth Meta-Address Registry
- x402 Protocol Whitepaper
- Coinbase x402 GitHub Repository
- dewiz-stealth-address GitHub Repository
- ScopeLift Stealth Address ERC Contracts
- EIP-3009: Transfer With Authorization (Circle)
- ERC-4337: Account Abstraction
- Liminal Cash — Stealth Address Thread
- ADR-001: Token Factory + Tokengentic REST API Integration

End of ADR-002